

Программирование и алгоритмизация

(МИСиС, осень-зима 2025)

Полевой Дмитрий Валерьевич
к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

как искать «быстро»?

Двоичный (бинарный) поиск

- он же
 - дихотомический поиск
 - поиск делением пополам
- осуществляется в отсортированном массиве
- сравнением искомого с «центральной»
- рекурсивный поиск в «подходящей» половине

Тонкости бинарного поиска

- пустой массив
- результат для «отсутствующего значения»
- переполнение в большом массиве
- на концах (первый/последний в массиве)
- повторяющиеся элементы
 - любой/первый/последний

Двоичное дерево поиска

(binary search tree)

- все поддеревья — деревья поиска
- для произвольного узла X значения ключей
 - в левом поддереве меньше
 - в правом поддерева больше (не меньше)

Самобалансирующиеся деревья

- AVL-дерево
- красно-черное дерево (red-black-tree)
- расширяющееся дерево (splay tree)
- и т.д.

когда можно искать
ещё «быстрее»?

Быстрое сравнение

- если сравнение элементов «медленное», то ускорение сравнения ускорит поиск
- сравнение «медленное» для «длинных»
 - строки
 - изображения, видео, звук
 - и т.д.

Хеширование

- детерминированный алгоритм (хэш функция) преобразования данных в битовую строку фиксированной длины
- хеш – битовая строка заданной длины
- коллизия – одинаковый хеш для разных данных

Основные свойства хэш-функции

- длинна (разрядность)
- вычислительная сложность
- распределение значений
- криптостойкость

Хорошая хеш-функция

- быстро вычисляется
- минимизирует количество коллизий
- идеальное хеширование
 - отображает каждый ключ в хэш без коллизий
 - уникальный идентификатор ключа, без сохранения какой-либо информации о ключе

Применение хеширования

- криптография
- контроль (целостности) данных
- быстрый поиск
 - «долгая» процедура сравнения элементов
 - «быстрый» алгоритм хеширования

Хеш-таблица

(структура данных)

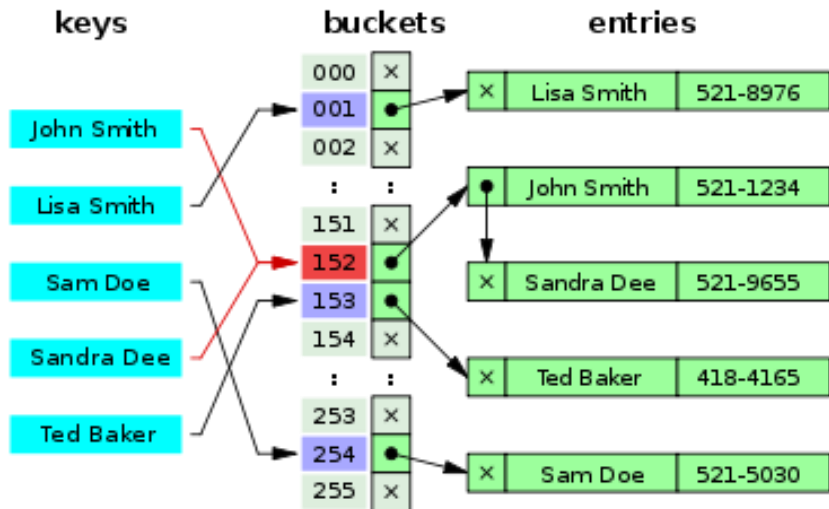
- хранение и поиск пар вида <ключ, значение>
- хорошо работает, если
 - хеш-ключи сравниваются быстрее объектов
 - коллизии редки

Поиск в хэш-таблице

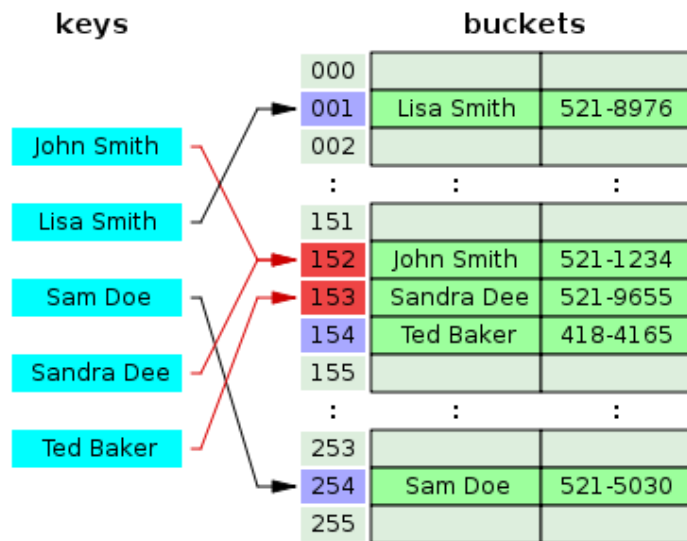
- хеширование
- проверка наличия значения в таблице
- разрешение коллизии (если есть)

Методы разрешения коллизий

Цепочки

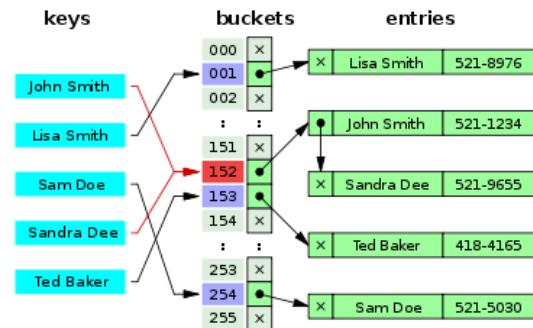


Открытая адресация



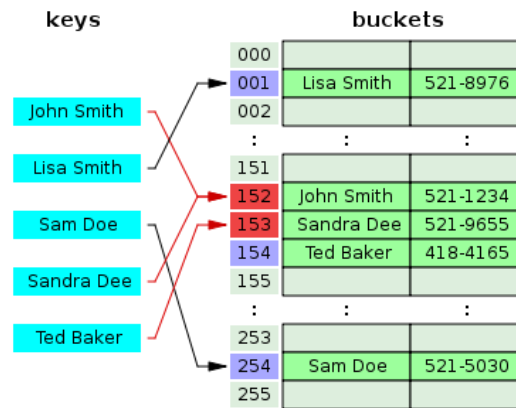
Разрешение коллизий (цепочки)

- пары хранятся в списке
- коллизия - список «удлинняется»
- поиск или удаление элемента
 - просмотр всех элементов цепочки
- «длинные цепочки» -> перестроить таблицу



Разрешение коллизий (открытая адресация)

- пары хранятся в массиве
- вставка/поиск используют последовательность проб
- удаление элемента – признак занятости
- «мало места» -> перестроить таблицу



Последовательности проб

- линейная
- квадратичная
- двойное хеширование

ваши вопросы

Операции над контейнерами и валидность итераторов

- операции над контейнерами могут делать итераторы невалидными
(зависти от реализации итераторов/контейнеров)
- гарантии безопасности описаны в стандарте
- всегда следите за валидностью итераторов
- избегайте хранения итераторов

Контейнеры переменного размера

- последовательность
(Sequence)
- ассоциативный контейнер
(Associative Container)

Последовательности в STL

- `vector<T>` – произвольный (массив)
- `deque<T>` – произвольный (дэк)
- `list<T>` – последовательный
двунаправленный (двусвязный список)
- `forward_list<T>` – последовательный
однонаправленный (односвязный список)

Заголовочные файлы STL

- совпадают с именем контейнера

`<array>`

`<vector>`

`<list>`

`<deque>`

`<forward_list>`

Вставка

- в конец (элемент)
`push_back`
- в начало (элемент)
`push_front`
- в позиции (элемент и интервал)
`insert`

Удаление

- из конца (элемент)
`pop_back()`
- из начала (элемент)
`pop_front()`
- в позиции (элемент и интервал)
`remove()`

Реверсивные итераторы для контейнеров

- значения интервала в обратном порядке
- специальные свойства и методы

`reverse_iterator`

`rbegin()` и `rend()`

`crbegin()` и `crend()`

- отличаются от обычных итераторов

reverse_iterator

- класс-адаптер для `iterator`
- для интервала `[f, l)` интервал `[reverse_iterator(l), reverse_iterator(f))` содержит значения в обратном порядке

- ОСНОВНОЕ ТОЖДЕСТВО

$$\&^*(\text{reverse_iterator}(i)) == \&^*(i - 1)$$
$$\&^*r == \&^*(r.\text{base}() - 1)$$

Доступ к элементам `std::list`

- через итератор (обобщенный вариант)
- с концов
- элементы произвольно расположены в памяти
- элементы не перемещаются

Размеры `std::list`

- `size_t size()`
число элементов (медленно!)
- `bool empty() const`
проверка на пустоту
- `size_t max_size() const`
максимальное число элементов (системно)

«Полезности» `std::list`

- `clear` - очистка
- `merge` - слияние отсортированных
- `splice` - перенос элементов
- `remove_if` – условное удаление
- `reverse` – инвертирование порядка
- `unique` – удаление повторяющихся
- `sort` - сортировка

`std::deque<>`

- блочно-списочная структура
- произвольный доступ
- быстрая вставка/удаление в конце и начале
- относительно быстрая вставка/удаление в середине
- изменение числа элементов делает итераторы невалидными

ваши вопросы

Заголовочные файлы STL

- **совпадают с именем контейнера**

`<set>` - в т.ч. `multiset`

`<map>` - в т.ч. `multimap`

`<unordered_set>` - в т.ч. `multiset`

`<unordered_map>` - в т.ч. `multimap`

- **ВСПОМОГАТЕЛЬНЫЕ**

`<utility>`

`<iterator>`

Контейнер `std::set`

- отсортированный, уникальные значений
- последовательно итерируется «в порядке»
- допускает параметризацию сравнения
- обычно реализуется сбалансированным деревом поиска

Контейнер `std::map<Key, Val>`

- отсортированный, ассоциативный
- последовательно итерируется «в порядке»
- хранит `pair<Key, Val>`
- допускает параметризацию сравнения
- обычно реализуется сбалансированным деревом поиска

Особенности operator[]

вставка элемента при необходимости

```
Val& operator[] (const Key& key);
```

```
Val& operator[] (Key&& key);
```

пример:

```
map<char, ptrdiff_t> counter;
```

```
// в цикле
```

```
counter[iCh] += 1;
```

Обращение к элементу с контролем

запускает исключение `std::out_of_range`

```
Val& at(const Key& key);
```

```
const Val& at(const Key& key) const;
```

`std::multimap`

- отсортированный, ассоциативный
- допускает повтор ключей
- пары для одного ключа в порядке вставки

Неупорядоченные ассоциативные контейнеры

- `std::unordered_set`
- `std::unordered_map`
- неотсортированные
- константное время операций
- обычно реализуется хеш-таблицей

ваши вопросы

А где в stl стек или очередь?

- адаптеры контейнеров

`std::stack`

`std::queue`

`std::priority_queue`

Адаптеры stl

- класс, преобразующий интерфейс адаптируемого типа к нужному интерфейсу
- в stl – шаблонный класс

`std::stack` (адаптер)

```
template<class T, class Cont = deque<T>>  
class stack;
```

```
bool empty() const  
T& top()  
const T& top() const  
void push(const T&)  
void pop()  
size_t size() const
```

std::queue (адаптер)

```
template<class T, class Cont =  
    deque<T>>  
class queue;  
  
bool empty() const  
size_t size() const  
T& front() // const  
T& back() // const  
void push(const T&)  
void pop()
```

`std::priority_queue` (адаптер)

```
template<class T, class Cont = vector<T>,  
        class Cmp =  
            less<typename Cont::value_type>>  
  
class priority_queue;
```

std::priority_queue (адаптер)

```
bool empty() const  
size_t size() const  
value_type& top() // const  
void push(const T&)  
void pop()
```

ваши вопросы

Оператор «запятая» (comma operator)

- бинарный
- вызывается в цепочке (a, b, c)
- встроенный обеспечивает порядок вычислений
- допускает перегрузку (не гарантирует порядка)

```
S& T::operator, (S& rhs);
```

```
S& operator, (const T& a, S& rhs);
```

Оператор «запятая»

- ассоциативность слева направо
(1,2,3,4,5) -> (((((1,2),3),4),5)
- самый низкий приоритет
- позволяет вычислить несколько выражений там, где синтаксис допускает одно

```
for (int i = 0, j = 10; i <= j; i += 1, j += 1)
//          ^list separator                ^comma op
```

Оператор «запятая» (пример использования)

- библиотека Eigen, инициализатор

Example:

```
Matrix3f m;  
m << 1, 2, 3,  
      4, 5, 6,  
      7, 8, 9;  
std::cout << m;
```

Output:

```
1 2 3  
4 5 6  
7 8 9
```

Оператор «запятая»

- может быть причиной «незаметных» ошибок
- следите за предупреждениями (warning)

пример планируемого

```
if (a.b) {
```

вариант с опечаткой

```
if (a,b) {
```

Что такое инициализатор?

Инициализатор

- начальное значение экземпляра
 - локальная переменная
 - глобальная переменная
 - передаваемое значение, в т.ч. умолчательное
 - умолчательное значение поля

Инициализатор в качестве передаваемого значения

```
//void set(Point pt = Point{});
```

```
set(Point(x, y));
```

```
set({x, y});
```

Инициализатор в качестве возвращаемого значения

```
Point get() {  
    return Point(x_, y_);  
}
```

```
Point get() {  
    return {x_, y_};  
}
```

Сколько вариантов
написать
инициализацию
вы знаете?

Инициализируем

```
// inits with 42
int i02 = 42;
int i03(42);
int i05{42};
int i06 = {42};
auto i09 = 42;
auto i12 = int{42};
```

Инициализируем

```
// inits with 0 - default value for int  
int    i04 = int();  
int    i07{};  
int    i08 = {};
```

Инициализируем

```
//C++11: std::initializer_list<int>, C++14: int  
auto i10{42};
```

```
//inits std::initializer_list<int> with 42  
auto i11 = {42};
```

```
//OK, inits int with 42 (comma operator)  
int i15 = (7, 42);  
auto i18 = (7, 42);
```

Инициализируем (с проблемами)

```
int i01;    //undefined value
```

```
int i13();  //declares a function
```

```
int i16 = int(7, 9); //CT error
```

```
int i17(7, 9);      //CT error
```

```
auto i19 = int(7, 9); //CT error
```

```
int i14(7, 9);      //CT error
```

А чё так сложно?

Длинная история и обратная совместимость C/C++98/C++11/C++14/C ++17/C++20/C++...

Инициализация по умолчанию

- «что не используется, за то не платим»
 - C/C++ - языки системного программирования
- локальные переменные и поля умолчательно не инициализируются
- обращение к неинициализированной переменной -> неопределенное поведение

Копирующая инициализация (copy initialization)

- инициализатор локальной переменной

```
int i = 0;
```

- передача по значению

```
int Square(const int i) {  
    return i * i;  
}
```

Копирующая инициализация

- если тип инициализатора не совпадает, то выполняется последовательность преобразования (conversion sequence)
- explicit конструкторы
 - не вызываются
 - игнорируются (т.е. может быть вызван менее подходящий не explicit конструктор)

Агрегатная инициализация

- используется для инициализации экземпляров агрегатных типов
 - массив (array)
 - структура (structure)
 - объединение (union)
- применяется копирующая инициализация для каждого элемента

Агрегатный тип

(aggregate type)

- «простая группа объектов без особенностей»
- RTFM (понятие меняется между стандартами)
 - наличие конструкторов
 - наследование
 - статические данные
 - и т.п.

Агрегатная инициализация (aggregate initialization)

// для массивов

```
int marks[4] = {2, 3, 4, 5};
```

```
int marks[] = {2, 3, 4, 5};
```

```
int ids[10] = {0};
```

// для структур

```
Complex zi = {0.0, 1.0};
```

```
Complex z1{1.0, 0.0};
```

Подагрегат (subaggregate)

- вложенный агрегатный тип
- может быть инициализирован через вложенные скобки
- обход подагрегатов производится рекурсивно

Пропуск скобок (brace elision)

```
struct W {  
    Complex c;  
    double pow = 0.0;  
};
```

`W w1 = {1.0, 1.0};` $\leftrightarrow$ `{{1.0, 1.0}, 0.0}`

`W w2 = {{1.0}, 1.0};` $\leftrightarrow$ `{{1.0, 0.0}, 1.0}`

Пропуск скобок (примеры)

```
int ar2d1[2][2] = {{1, 2}, {3, 4}};
```

```
// brace elision: {1, 2} {3, 4}
```

```
int ar2d2[2][2] = {1, 2, 3, 4};
```

```
// only first column: {1, 0}, {2, 0}
```

```
int ar2d3[2][2] = {{1}, {2}};
```

Инициализация нелокальных объектов

- статическая и динамическая инициализация, которая происходит до входа в `main`
- статические объекты всегда инициализируются
 - константой (пользователь, на этапе компиляции)
 - нуль-инициализатор (zero-initialization)
- повышенное внимание при наличии зависимостей, внешней компоновки и `inline`

Инициализация нелокальных объектов

- инициализаторы/константы этапа компиляции
 - литералы
 - `constexpr` и `constexpr` (в т.ч. конструкторы)
 - `constexpr` (для проверки)
- избегать умолчательности

ваши вопросы

Прямая инициализация (direct-initialization)

- при использовании ()
 - `T obj (arg) ;`
 - `T (arg)`
 - `static_cast<T> (arg)`
 - `new T (args, ... )`
 - `T::T () : x_ (arg)`
 - `[arg] () { ... }`

Прямая инициализация

- допускает много параметров
- конструкторы ()
 - не выполняется последовательность преобразования
 - происходит разрешение перегрузки
- может путаться с объявлением функции

Универсальная инициализация (uniform initialization)

- **прямая инициализация списком**

```
// direct-list-initialization
```

```
Rational r{1, 2};
```

- **копирования**

```
// copy-list-initialization
```

```
Rational r = {1, 2};
```

Список инициализаторов (braced-init-list)

- не имеет типа
- не является объектом
- не допускает преобразования, сужающие диапазон значений (narrowing conversions)
- м.б. скрыто преобразован в `std::initializer_list`

`std::initializer_list<>`

- специальный тип
- используется для перегрузки конструкторов

`V(std::initializer_list<T> data)`

- вектор фиксированного размера
- элементы `const`
- доступ через итераторы и `begin()`/`end()`

Неудачность `initializer_list`

- выигрывает перегрузки у других `ctor`
 - может вызывать большое удивление
 - заставляет больше помнить
- потенциально приводит к копированию данных
- препятствует перемещению параметров из списка
- пропуск скобок не работает

Путаница с `initializer_list`

```
// вектор содержит 0, 0, 0  
std::vector<int> v(3, 0);
```

```
// вектор содержит 3, 0  
std::vector<int> v{3, 0};
```

Путаница с `initializer_list`

```
std::string s(48, 'a'); // "aaa...aaa"  
std::string s{48, 'a'}; // "0a"
```

Особенности списка инициализаторов

- пропуск скобок не работает -> неочевидная интерпретация вложенных фигурные скобки
- пустые фигурные скобки ведут себя иначе, чем не-пустые
- `auto` работает не всегда очевидным образом
- ломает макросы (запятая внутри скобок)
- плохо совместим с шаблонами

Назначенная инициализация (designated initializers)

```
struct A { int x; int y; int z; };
```

```
// error; designator order does not match  
declaration order A
```

```
A a{.y = 2, .x = 1};
```

```
// ok, b.y initialized to 0
```

```
b{.x = 1, .z = 2};
```

ваши вопросы

Инициализация (рекомендации)

- всегда инициализировать
- для простых типов – копированием =
- для вызова конструктора – ()
- для временных объектов – { }

Инициализация (рекомендации)

= {args} и = {}

- для агрегатной инициализации
- конструирование
от `std::initializer_list`
- для direct member initialization,
т.к. с ними нельзя использовать (args)

Инициализация (рекомендации)

- умолчательные инициализаторы полей
(direct member initializers)

```
struct Complex {  
    double re = 0.0;  
    double im = 0.0;  
};
```

Инициализация (рекомендации)

- СПИСОК инициализации

```
Complex(const double r, const double i)  
    : re(r)  
    , im(i) {  
  
}
```

Выбирайте момент копирования

```
T operator+(const T& lhs, const T& rhs) {  
    return T(lhs) += rhs;  
}
```

```
T operator+(T lhs, const T& rhs) {  
    return lhs += rhs;  
}
```

ваши вопросы

Days 1 - 10
Teach yourself variables, constants, arrays, strings, expressions, statements, functions,...



Days 11 - 21
Teach yourself program flow, pointers, references, classes, objects, inheritance, polymorphism,



Days 22 - 697
Do a lot of recreational programming. Have fun hacking but remember to learn from your mistakes.



Days 698 - 3648
Interact with other programmers. Work on programming projects together. Learn from them.



Days 3649 - 7781
Teach yourself advanced theoretical physics and formulate a consistent theory of quantum gravity.



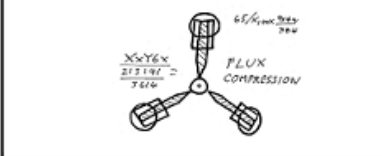
Days 7782 - 14611
Teach yourself biochemistry, molecular biology, genetics,...



Day 14611
Use knowledge of biology to make an age-reversing potion.



Day 14611
Use knowledge of physics to build flux capacitor and go back in time to day 21.



Day 21
Replace younger self.



As far as I know, this
is the easiest way to
"Teach Yourself C++ in 21 Days".